



Anexo 1 — default, null y Empty en C# (Parte 2 de 2)

Curso: Arquitectura de Software Profesional con Clean Architecture, DDD y SOLID
*Complemento del **Módulo 1 — Entidades** · Prerrequisito del **Módulo 2 — Value Objects***

Contenido de esta parte

Tema 7	Errores comunes
Tema 8	Qué NO hacer — código incorrecto
Tema 9	Ejemplo práctico integrado
Tema 10	Actividades prácticas

Tema 7 — Errores comunes

Error 1 — Usar `IsEmpty` cuando se necesita `IsWhiteSpace`

Es el error más frecuente en validaciones de dominio. `IsEmpty` no detecta cadenas compuestas solo por espacios, lo que permite que nombres como " " pasen la validación y queden guardados en la base de datos.

```
1 // ✗ MAL: IsEmpty no detecta " " (solo espacios)
2 if (string.IsNullOrEmpty(fullName))
3     throw new ArgumentException("El nombre es obligatorio.");
4 // → Customer.Create(" ", ...) pasa la validación → nombre inválido guardado
5
6 // ✓ BIEN: IsWhiteSpace cubre null, "" y " "
7 if (string.IsNullOrWhiteSpace(fullName))
8     throw new ArgumentException("El nombre es obligatorio.", nameof(fullName));
```

Error 2 — Acceder a `.Value` de un nullable sin verificar

Acceder directamente a `.Value` de un tipo nullable sin verificar primero que tenga valor provoca `InvalidOperationException` en tiempo de ejecución. El compilador no lo detecta como error, lo que lo hace especialmente peligroso.

```
1 // ✗ MAL: acceder a .Value sin verificar – lanza InvalidOperationException en runtime
2 DateTime? modificadoEn = null;
3 string texto = modificadoEn.Value.ToString(); // ✗ explota en runtime
4
5 // ✓ BIEN: verificar antes con HasValue
6 if (modificadoEn.HasValue)
7     string texto = modificadoEn.Value.ToString();
8
9 // ✓ BIEN: usar operador ?? para dar valor por defecto
10 string texto = (modificadoEn ?? DateTime.UtcNow).ToString();
```

Error 3 — Confundir default de string con `string.Empty`

En C#, `default(string)` es `null`, no `string.Empty`. Muchos principiantes asumen que una variable `string` sin inicializar tiene el valor `""`, cuando en realidad tiene `null`.

```
1 // ✗ ERROR CONCEPTUAL: asumir que string no inicializado = ""
2 string nombre; // ← default = null, NO es ""
3 if (nombre == string.Empty) { } // ✗ NullReferenceException: nombre es null
4
5 // ✓ BIEN: inicializar explícitamente para evitar null inesperado
6 string nombre = string.Empty; // explícito y seguro
7
```

```
8 // ✅ BIEN: usar IsNullOrWhiteSpace que maneja ambos casos
9 if (string.IsNullOrWhiteSpace(nombre)) { } // seguro con null o ""
```

Error 4 — Confundir Guid.Empty con "no existe"

`Guid.Empty` no significa "el ID no existe": significa "el Guid tiene todos ceros". Usarlo como si fuera `null` puede llevar a guardar registros con un ID inválido que podría colisionar si más de un registro tiene el mismo `Guid.Empty` como ID.

```
1 // ❌ MAL: usar Guid.Empty como ID real de una entidad
2 Guid customerId = Guid.Empty; // ← no es un ID válido para una entidad de dominio
3 _repository.Save(customerId); // guarda un registro con ID = todos ceros
4
5 // ✅ BIEN: el ID siempre lo genera BaseEntity con Guid.NewGuid()
6 // Guid.Empty en BaseEntity se usa SOLO para campos de auditoría "sin asignar"
7 CreatedBy = Guid.Empty; // ✅ "ningún usuario asignado aún" – no es el ID de la entidad
```

Error 5 — No usar nameof() en las excepciones

Escribir el nombre del parámetro como un string literal en la excepción es un error silencioso: si el parámetro se renombra, el mensaje de error queda desactualizado y el diagnóstico del bug se vuelve más difícil.

```
1 // ❌ MAL: nombre del parámetro como string literal – se desactualiza al refactorizar
2 throw new ArgumentException("El nombre es obligatorio.", "fullName");
3
4 // ✅ BIEN: nameof() es refactor-safe – si el parámetro se renombra, actualiza solo
5 throw new ArgumentException("El nombre es obligatorio.", nameof(fullName));
```

Tema 8 — Qué NO hacer

```
1 // =====
2 // ✖ QUÉ NO HACER – método Create() con validaciones incorrectas
3 // Reúne todos los errores comunes del Tema 7 en un solo ejemplo
4 // =====
5 public static Customer Create(string fullName, string email, DateTime? birthDate)
6 {
7     // ✖ Error 1: IsNullOrEmpty no detecta " " (solo espacios)
8     if (string.IsNullOrEmpty(fullName))
9         throw new ArgumentException("El nombre es obligatorio.");
10
11     // ✖ Error 2: comparar string con "" en lugar de usar IsNullOrWhiteSpace
12     if (email == "")
13         throw new ArgumentException("El email es obligatorio.");
14     // → si email es null → NullReferenceException antes de llegar al throw
15     // → si email es " " → pasa sin error
16
17     // ✖ Error 3: acceder a .Value sin verificar HasValue
18     var year = birthDate.Value.Year; // ✖ InvalidOperationException si birthDate es null
19
20     // ✖ Error 4: asignar Guid.Empty como ID de la entidad
21     return new Customer
22     {
23         Id = Guid.Empty, // ✖ ID inválido – todos los clientes tendrían el mismo
24         FullName = fullName, // ✖ sin Trim() – espacios al inicio/final quedan
25         Email = email // ✖ sin ToLower() – "JUAN@mail.com" ≠ "juan@mail.com"
26     };
27 }
```

Tema 9 — Ejemplo práctico integrado

El siguiente ejemplo muestra cómo aplicar correctamente todo lo aprendido en el contexto del caso de uso **CU-CLI-001 — Alta de Cliente**. Se trata de un método auxiliar de validación privado que centraliza todos los controles, usando las técnicas correctas para cada tipo de dato.

Archivo: CommerX.Domain / Customers / Entities / Customer.cs
Fragmento: validaciones del método Create() – versión correcta

```
1 // =====
2 // ✅ CORRECTO – validaciones completas con las técnicas de este anexo
3 // Cada control usa el método adecuado para su tipo de dato
4 // =====
5 public static Customer Create(
6     string? fullName,
7     string? lastName,
8     string? document,
9     string? email,
10    string? phone,
11    string? address,
12    DateTime birthDate)
13 {
14     // — Validaciones de strings —
15     // IsNullOrWhiteSpace cubre: null, "" y " " en una sola llamada
16     if (string.IsNullOrWhiteSpace(fullName))
17         throw new ArgumentException("El nombre es obligatorio.", nameof(fullName));
18
19     if (string.IsNullOrWhiteSpace(lastName))
20         throw new ArgumentException("El apellido es obligatorio.", nameof(lastName));
21
22     if (string.IsNullOrWhiteSpace(document))
23         throw new ArgumentException("El documento es obligatorio.", nameof(document));
24
25     if (string.IsNullOrWhiteSpace(email))
26         throw new ArgumentException("El email es obligatorio.", nameof(email));
27
28     // — Validación de DateTime — no es nullable, se compara con default —
29     // Si birthDate es default (01/01/0001) → nadie pasó una fecha real
30     if (birthDate == default)
31         throw new ArgumentException("La fecha de nacimiento es obligatoria.",
32             nameof(birthDate));
33
34     // — Regla de negocio: mayor de 18 años —
35     // birthDate es DateTime (no nullable) → .Date es seguro, no necesita .HasValue
36     var today = DateTime.Today;
37     var age = today.Year - birthDate.Year;
38     if (birthDate.Date > today.AddYears(-age)) age--;
39     if (age < 18)
```

```
39         throw new ArgumentException("El cliente debe ser mayor de 18 años.",
40         nameof(birthDate));
41
42         // — Construcción — solo se llega aquí si TODO pasó —————
43         // new Customer() → base() → BaseEntity genera Id y CreatedOn automáticamente
44         return new Customer
45         {
46             // Trim() elimina espacios al inicio y al final — fullName pasó IsNullOrWhiteSpace
47             // por lo que aquí ya sabemos que no es null → Trim() es seguro
48             FullName = fullName.Trim(),
49             LastName = lastName.Trim(),
50             Document = document.Trim(),
51             // ToLower(): emails siempre en minúsculas para comparaciones consistentes
52             Email = email.Trim().ToLower(),
53             // phone y address son opcionales: si son null → usar string.Empty
54             Phone = phone?.Trim() ?? string.Empty,
55             Address = address?.Trim() ?? string.Empty,
56             // .Date: elimina el componente de hora — solo guardamos la fecha de nacimiento
57             BirthDate = birthDate.Date
58         };
59     }
```

Puntos clave del ejemplo integrado:

- `string?` en los parámetros: se declaran como nullable para que el compilador permita recibir null y nosotros lo controlemos explícitamente.
- Después de `IsNullOrWhiteSpace`, el compilador sabe que la variable no es null → `.Trim()` es seguro sin necesidad de `?.`
- `phone?.Trim() ?? string.Empty`: si phone es null → devuelve string.Empty; si existe → aplica Trim(). Telefono y dirección son opcionales en este diseño.
- `birthDate == default`: detecta si nadie pasó una fecha real (01/01/0001), sin necesitar que sea nullable.
- `birthDate.Date`: elimina la hora — solo importa la fecha para calcular la edad.

Tema 10 — Actividades prácticas

Actividades contextualizadas en **CU-CLI-001 — Alta de Cliente.**

Actividad 1 — Predecir resultados

Consigna:

Sin ejecutar código, indicar qué devuelve cada expresión y por qué:

<code>string.IsNullOrEmpty(null)</code>	
<code>string.IsNullOrEmpty("")</code>	
<code>string.IsNullOrEmpty(" ")</code>	
<code>string.IsNullOrWhiteSpace(" ")</code>	
<code>string.IsNullOrWhiteSpace("Juan")</code>	
<code>default(Guid) == Guid.Empty</code>	
<code>default(DateTime) == DateTime.MinValue</code>	
<code>default(string) == null</code>	
<code>default(string) == string.Empty</code>	
<code>default(int)</code>	

Actividad 2 — Detectar el bug

Consigna:

El siguiente método tiene al menos 4 errores relacionados con null, default y Empty. Identificarlos y corregirlos:

```
public static Customer Create(string fullName, string email, DateTime? birthDate)
{
    if (fullName == "")
        throw new ArgumentException("El nombre es obligatorio.");

    if (email == null)
        throw new ArgumentException("El email es obligatorio.");

    var year = birthDate.Value.Year;
    if (DateTime.Now.Year - year < 18)
        throw new ArgumentException("Debe ser mayor de edad.");
}
```

```
return new Customer
{
    Id      = Guid.Empty,
    FullName = fullName,
    Email   = email
};
}
```

Preguntas:

1. ¿Qué pasa si se llama con `fullName = " "`? ¿Pasa la validación?
2. ¿Qué excepción lanza `birthDate.Value.Year` si `birthDate` es null?
3. ¿Por qué `Id = Guid.Empty` es incorrecto para una entidad?
4. ¿Por qué el cálculo de edad con solo `Year` es incorrecto? Dar un ejemplo.

Actividad 3 — Completar el método**Consigna:**

Completar los espacios en blanco con la expresión correcta:

```
public static Customer Create(string? fullName, string? email, DateTime birthDate)
{
    // validar que fullName no sea null, vacío ni solo espacios
    if ( _____(fullName) )
        throw new ArgumentException("El nombre es obligatorio.", _____(fullName));

    // validar que email no sea null, vacío ni solo espacios
    if ( _____(email) )
        throw new ArgumentException("El email es obligatorio.", _____(email));

    // validar que birthDate no sea el valor por defecto
    if (birthDate == _____)
        throw new ArgumentException("La fecha es obligatoria.", nameof(birthDate));

    // asignar phone como opcional: si es null → usar _____
    Phone = phone?._____() ?? _____;
}
```

Actividad 4 — Análisis de DateTime?**Consigna:**

Responder las siguientes preguntas sobre el diseño de `BaseEntity`:

1. ¿Por qué `ModifiedOn` es `DateTime?` y no `DateTime`? ¿Qué significaría semánticamente si fuera `DateTime` con valor `DateTime.MinValue`?
2. Si quisieras mostrar en la UI "Última modificación: nunca" cuando no fue modificado, ¿cómo verificarías si `ModifiedOn` es null? Escribir el código.
3. ¿Es correcto usar `ModifiedOn.Value.ToString()` directamente? ¿Cuándo lanzaría excepción y cuál?

4. Escribir la expresión que devuelve `ModifiedOn` como string, usando `"N/A"` como valor por defecto si es null.

Actividad 5 — Decisión de diseño

Consigna:

Para cada campo de `Customer`, decidir si debe ser `string`, `string?`, o inicializarse con `string.Empty`. Justificar la decisión:

Campo	Tipo elegido	Justificación
<code>FullName</code> – obligatorio		
<code>Email</code> – obligatorio		
<code>Phone</code> – opcional		
<code>Address</code> – opcional		
<code>MiddleName</code> – puede no existir		

Pregunta de reflexión:

- ¿Qué diferencia hay entre declarar una propiedad como `string?` y declararla como `string` inicializada con `string.Empty`? ¿Cuándo conviene cada una?

✓ Al completar este anexo, el alumno puede identificar con precisión cuándo un valor es `null`, `default` o `Empty`, qué excepción provoca cada caso y cómo detectarlos correctamente en validaciones de dominio.

▶ Continúa con el **Módulo 2 — Value Objects**, donde estas técnicas se aplican en cada `Create()` de los Value Objects.